



CRYPTOGRAPHY

Archana M

Department of Computer Applications

CRYPTOGRAPHY

Hash Functions and Digital Signature- Unit 4:

Archana M

Department of Computer Applications

Introduction to Cryptographic Hash Functions

- A cryptographic hash function takes a message of arbitrary length and creates a message digest of fixed length. i.e., A **cryptographic hash function** is a mathematical function used in cryptography that takes an input (called a *message*) of any size and produces a fixed-size output called a **hash value** or **digest**.
- Example:
 - Input: any data (text, file, password, image, etc.)
 - Output: a fixed-length string (hash)
 - Even a tiny change in input gives a completely different hash.

Key Properties of Cryptographic Hash Functions

1. Deterministic

- Same input → same hash every time

2. Fixed Output Length

- No matter input size, output size is constant (e.g., 256 bits for SHA-256)

3. Pre-image Resistance

- Given a hash, it's practically impossible to find the original input

4. Second Pre-image Resistance

- Hard to find another input with the same hash

5. Collision Resistance

- Very hard to find two different inputs with the same hash

6. Avalanche Effect

- Small change in input → huge change in output

Applications

1. Password Storage

- Store hash instead of actual password

2. Data Integrity

- Verify file hasn't changed

3. Digital Signatures

- Hash is signed instead of full message

4. Blockchain

- Used in systems like Bitcoin

5. Message Authentication

- Used in HMAC (Hash-based Message Authentication Code)

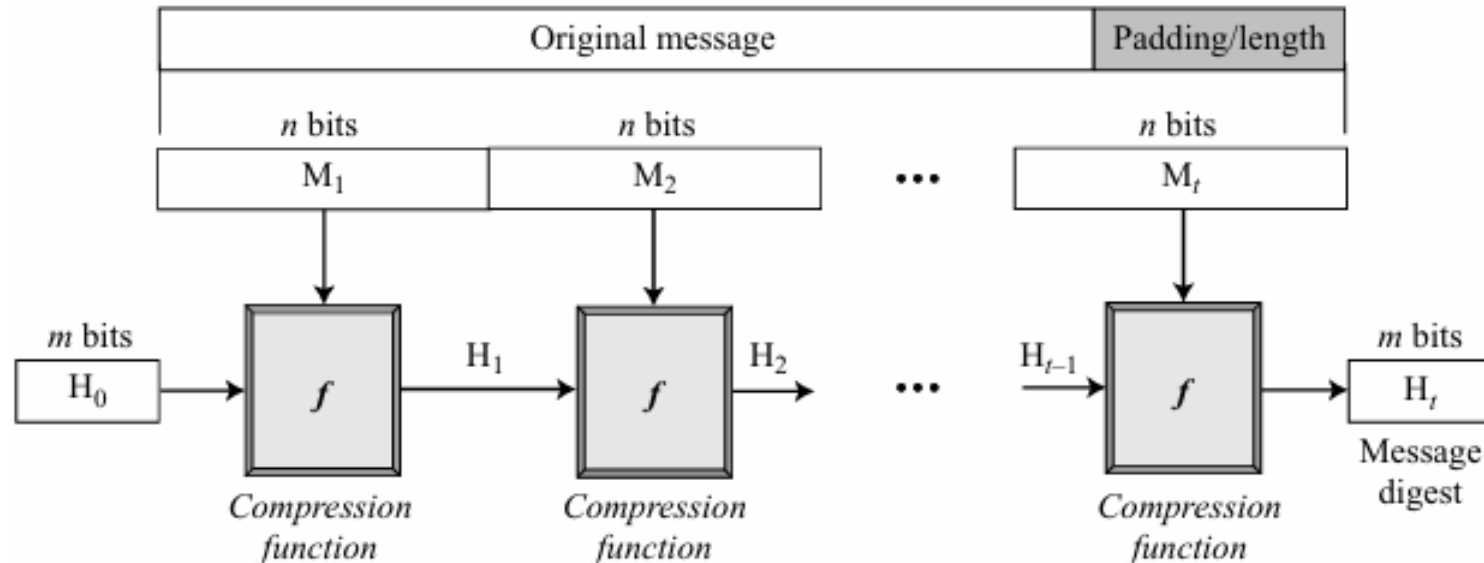
Iterated Hash Function

- All cryptographic hash functions need to create a fixed-size digest out of a variable-size message.
- Creating such a function is best accomplished using iteration.
- Instead of using a hash function with variable-size input, a function with fixed-size input is created and is used a necessary number of times.
- The fixed-size input function is referred to as a compression function.
- It compresses an n -bit string to create an m -bit string where n is normally greater than m .
- The scheme is referred to as an iterated cryptographic hash function

Merkle-Damgard Scheme

- The Merkle-Damgard scheme is an iterated hash function that is collision resistant if the compression function is collision resistant. This can be proved

Figure 12.1 *Merkle-Damgard scheme*



The scheme uses the following steps:

1. The message length and padding are appended to the message to create an augmented message that can be evenly divided into blocks of n bits, where n is the size of the block to be processed by the compression function.
2. The message is then considered as t blocks, each of n bits. We call each block $M_1, M_2, \dots, M_t$. We call the digest created at t iterations $H_1, H_2, \dots, H_t$.
3. Before starting the iteration, the digest H_0 is set to a fixed value, normally called IV (initial value or initial vector).
4. The compression function at each iteration operates on H_{i-1} and M_i to create a new H_i . In other words, we have $H_i = f(H_{i-1}, M_i)$, where f is the compression function.
5. H_t is the cryptographic hash function of the original message, that is, $h(M)$.

Two Groups of Compression Functions

- The Merkle-Damgard scheme is the basis for many cryptographic hash functions today.
- The only thing we need to do is design a compression function that is collision resistant and insert it in the Merkle-Damgard scheme. There is a tendency to use two different approaches in designing a hash function.
- In the first approach, the compression function is made from scratch: it is particularly designed for this purpose.
- In the second approach, a symmetric-key block cipher serves as a compression function.

CRYPTOGRAPHY

Hash Functions and Digital Signature- Unit 4:

Archana M

Department of Computer Applications

Message Digest (MD)

- Several hash algorithms were designed by Ron Rivest. These are referred to as MD2, MD4, and MD5, where MD stands for Message Digest.
- The last version, MD5, is a strengthened version of MD4 that divides the message into blocks of 512 bits and creates a 128-bit digest.
- It turned out that a message digest of size 128 bits is too small to resist collision attack.

MD5- Steps

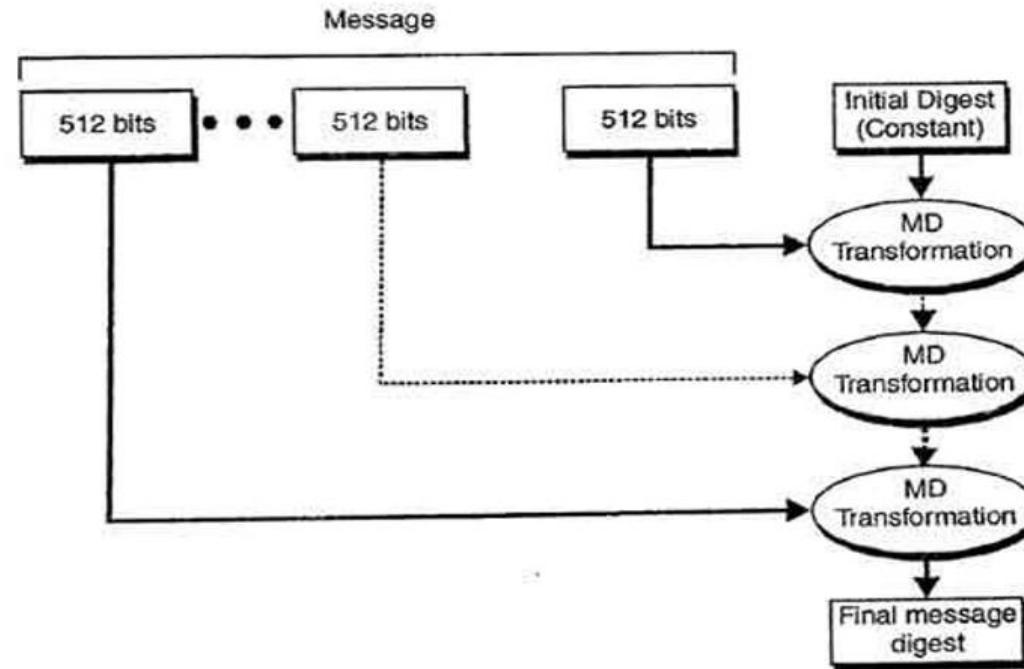
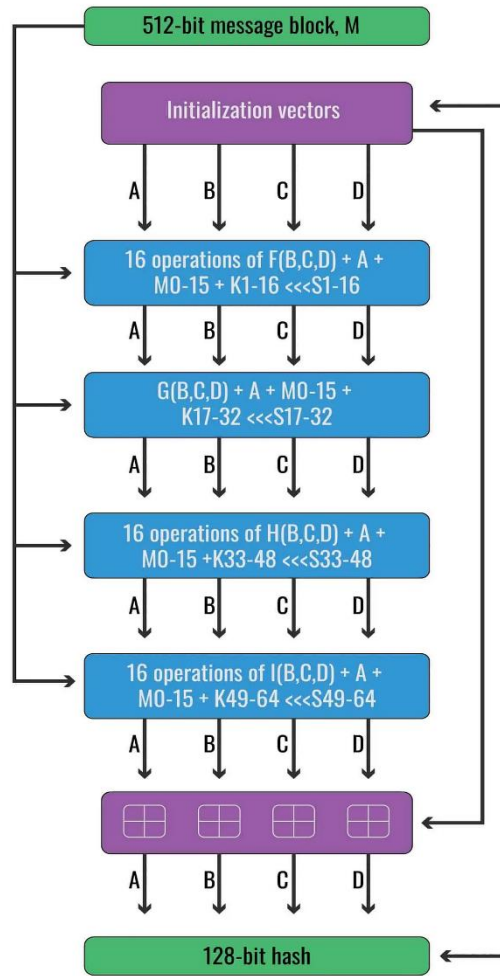
Input → any data (text, file, password)

Output → **128-bit hash** (usually shown as 32 hexadecimal characters)

Example:

Input: hello

MD5: 5d41402abc4b2a76b9719d911017c592



Generation of message digest using MD5.

Step 1: Padding :Message is padded to make its length a multiple of 512 bits

Step 2: Divide into Blocks :Split into 512-bit chunks

Step 3: Initialize Buffers :4 internal variables (A, B, C, D)

Step 4: Compression Function :Each block goes through 64 operations (rounds)

Step 5: Output :Final result → 128-bit hash

Key Properties

- Deterministic (same input → same output)
- Fast to compute
- Fixed output size (128 bits)
- Avalanche effect (small change → big difference)

MD5 is NOT Secure

- MD5 is now considered **broken** due to:
 - Collisions : Two different inputs can produce the **same hash**
 - Vulnerable to Attacks : Collision attacks , and Preimage weaknesses (partially)

Secure Hash Algorithm (SHA)



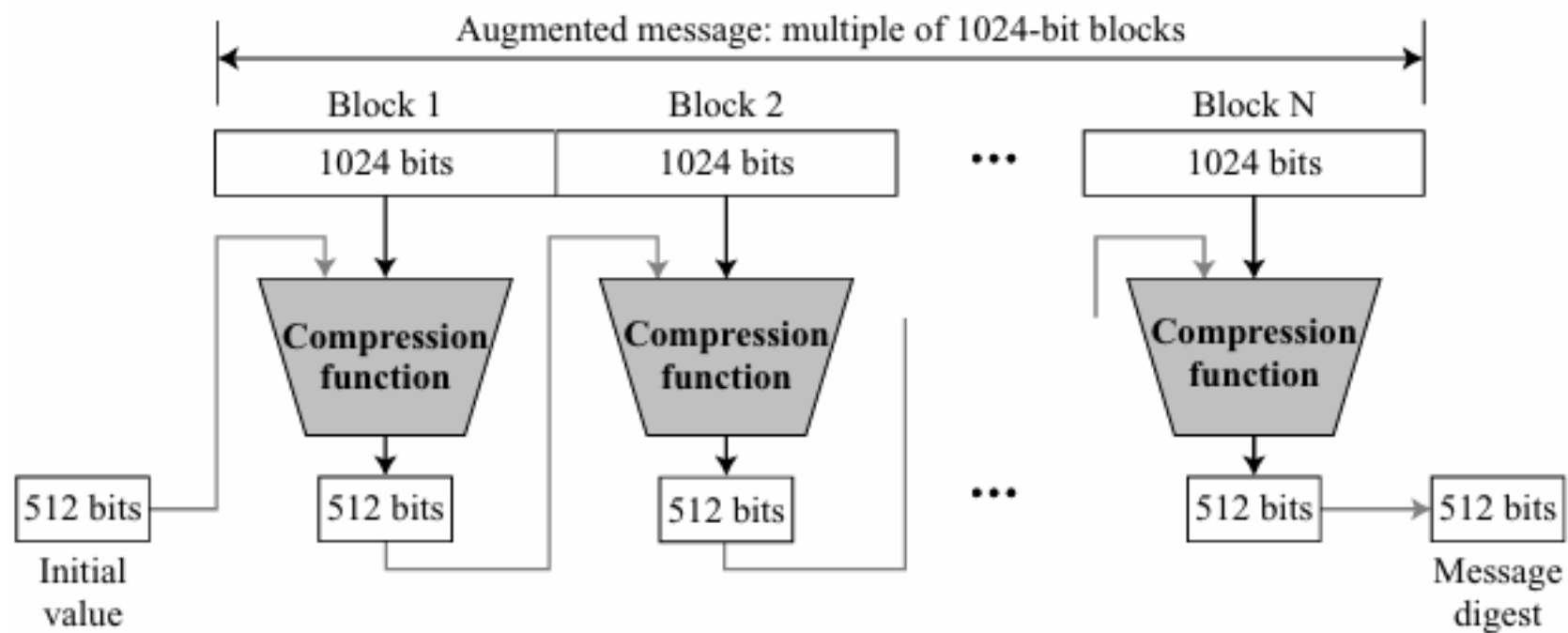
- The Secure Hash Algorithm (SHA) is a standard that was developed by the National Institute of Standards and Technology (NIST) and published as a Federal Information Processing standard (FIP 180).
- It is sometimes referred to as Secure Hash Standard (SHS).
- The standard is mostly based on MD5. The standard was revised in 1995 under FIP 180-1, which includes SHA-1
- It was revised later under FIP 180-2, which defines four new versions: SHA-224, SHA-256, SHA-384, and SHA-512

Table 12.1 *Characteristics of Secure Hash Algorithms (SHAs)*

<i>Characteristics</i>	<i>SHA-1</i>	<i>SHA-224</i>	<i>SHA-256</i>	<i>SHA-384</i>	<i>SHA-512</i>
Maximum Message size	$2^{64} - 1$	$2^{64} - 1$	$2^{64} - 1$	$2^{128} - 1$	$2^{128} - 1$
Block size	512	512	512	1024	1024
Message digest size	160	224	256	384	512
Number of rounds	80	64	64	80	80
Word size	32	32	32	64	64

-
- SHA-512 is the version of SHA with a 512-bit message digest
 - This version, like the others in the SHA family of algorithms, is based on the Merkle-Damgard scheme. It has a more complex structure than the others, and its message digest is the longest.
 - SHA-512 creates a digest of 512 bits from a multiple-block message. Each block is 1024 bits in length, as shown in Figure 12.6.

Figure 12.6 *Message digest creation SHA-512*



-
- The digest is initialized to a predetermined value of 512 bits.
 - The algorithm mixes this initial value with the first block of the message to create the first intermediate message digest of 512 bits.
 - This digest is then mixed with the second block to create the second intermediate digest.
 - Finally, the $(N - 1)$ th digest is mixed with the Nth block to create the Nth digest.
 - When the last block is processed, the resulting digest is the message digest for the entire message.

Message Preparation

- SHA-512 insists that the length of the original message be less than 2^{128} bits.
- This means that if the length of a message is equal to or greater than 2^{128} , it will not be processed by SHA-512.
- This is not usually a problem because 2^{128} bits is probably larger than the total storage capacity of any system

Example 12.1

This example shows that the message length limitation of SHA-512 is not a serious problem. Suppose we need to send a message that is 2^{128} bits in length. How long does it take for a communications network with a data rate of 2^{64} bits per second to send this message?

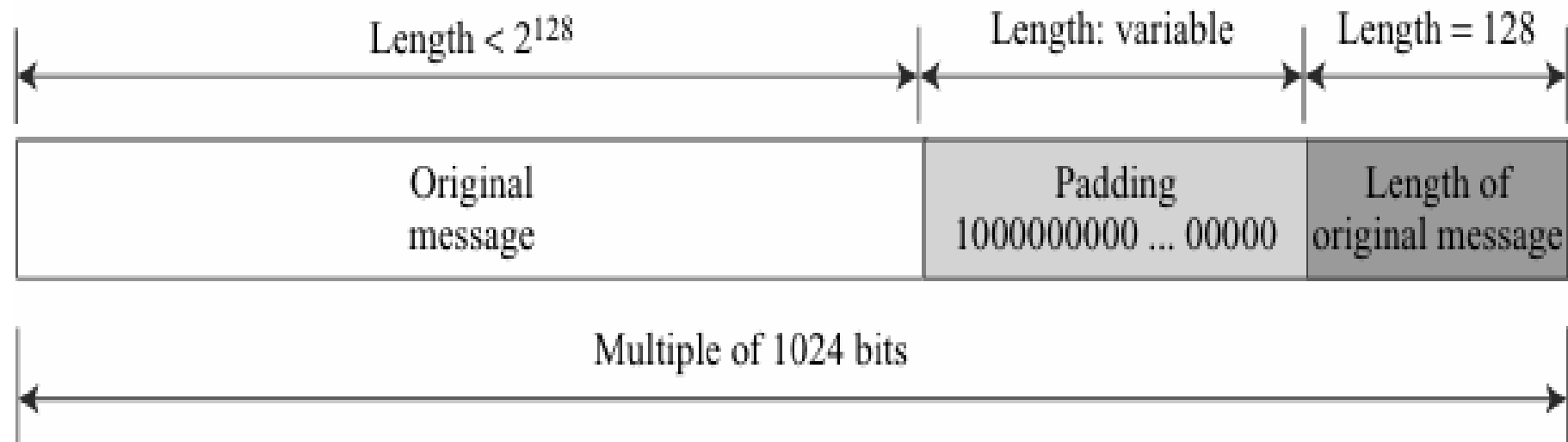
Solution

A communications network that can send 2^{64} bits per second is not yet available. Even if it were, it would take many years to send this message. This tells us that we do not need to worry about the SHA-512 message length restriction.

Length Field and Padding

- Before the message digest can be created, SHA-512 requires the addition of a 128-bit unsigned-integer length field to the message that defines the length of the message in bits.
- This is the length of the original message before padding.
- An unsigned integer field of 128 bits can define a number between 0 and $2^{128} - 1$.
- which is the maximum length of the message allowed in SHA-512. The length field defines the length of the original message before adding the length field or the padding

Figure 12.7 *Padding and length field in SHA-512*



Before the addition of the length field, we need to pad the original message to make the length a multiple of 1024. We reserve 128 bits for the length field, as shown in Figure 12.7. The length of the padding field can be calculated as follows. Let $|M|$ be the length of the original message and $|P|$ be the length of the padding field.

$$(|M| + |P| + 128) = 0 \bmod 1024 \quad \rightarrow \quad |P| = (-|M| - 128) \bmod 1024$$

The format of the padding is one 1 followed by the necessary number of 0s.

Example 12.3

What is the number of padding bits if the length of the original message is 2590 bits?

Solution

We can calculate the number of padding bits as follows:

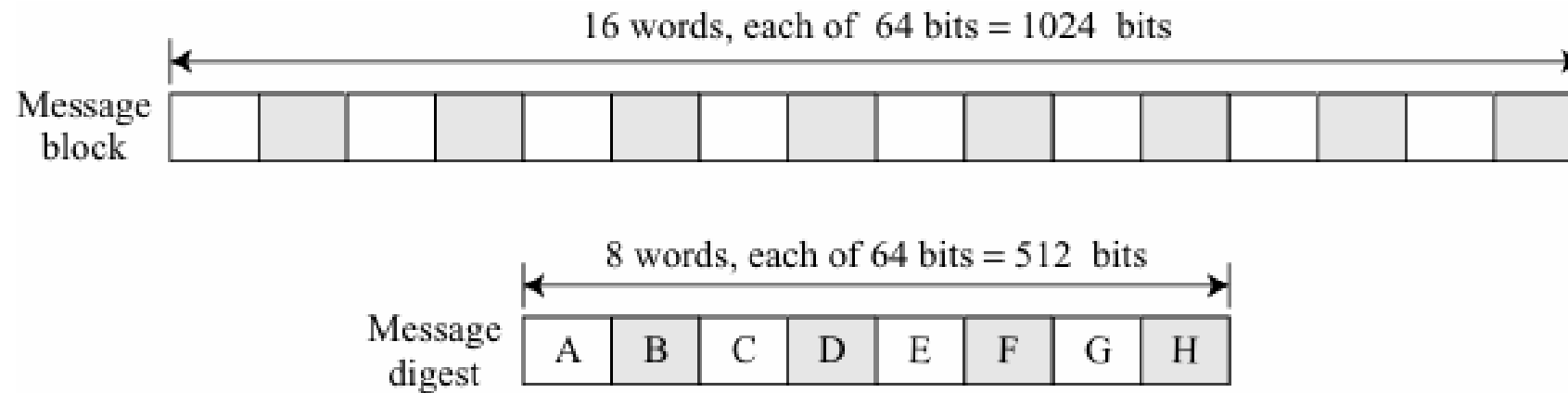
$$|P| = (-2590 - 128) \bmod 1024 = -2718 \bmod 1024 = 354$$

The padding consists of one 1 followed by 353 0's.

Words

- SHA-512 operates on words;
- it is word oriented. A word is defined as 64 bits.
- This means that, after the padding and the length field are added to the message, each block of the message consists of sixteen 64-bit words.
- The message digest is also made of 64-bit words, but the message digest is only eight words and the words are named A, B, C, D, E, F, G, and H, as shown in Figure

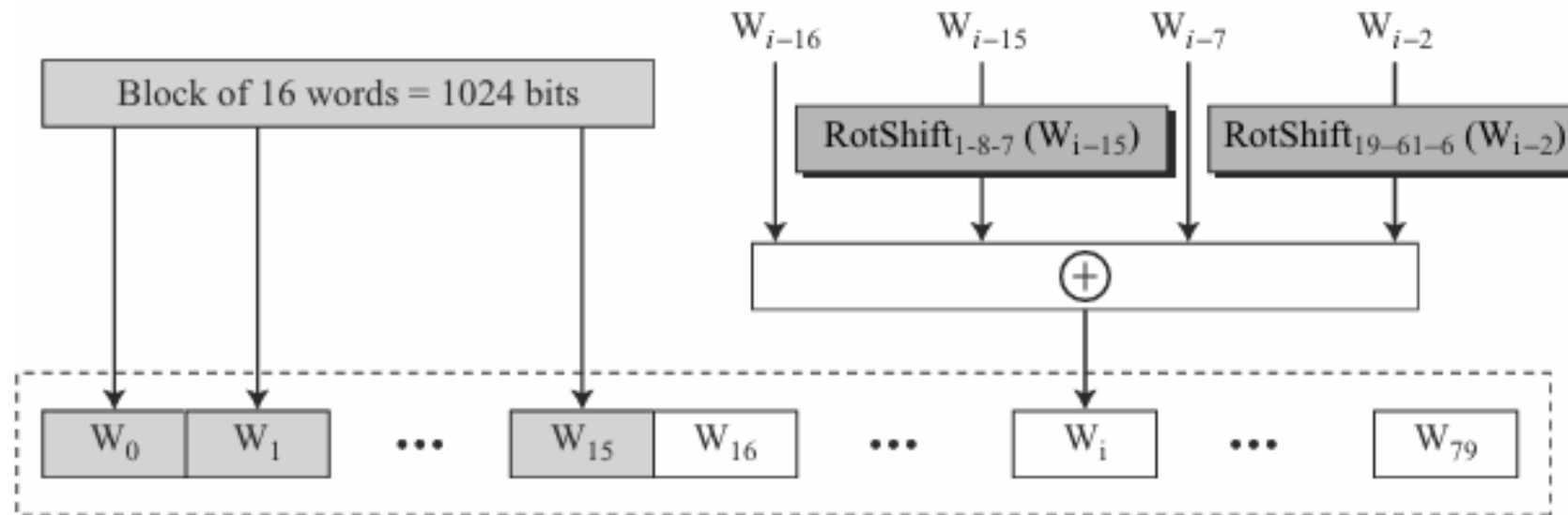
Figure 12.8 *A message block and the digest as words*



Word Expansion

- Before processing, each message block must be expanded. A block is made of 1024 bits, or sixteen 64-bit words.
- As we will see later, we need 80 words in the processing phase. So the 16-word block needs to be expanded to 80 words, from W .
- Figure 12.9 shows the word-expansion process.
- The 1024-bit block becomes the first 16 words;
- the rest of the words come from already-made words according to the operation shown in the figure.

Figure 12.9 *Word expansion in SHA-512*



$\text{RotShift}_{1-m-n}(x)$: $\text{RotR}_l(x) \oplus \text{RotR}_m(x) \oplus \text{ShL}_n(x)$

$\text{RotR}_i(x)$: Right-rotation of the argument x by i bits

$\text{ShL}_i(x)$: Shift-left of the argument x by i bits and padding the left by 0's.

Message Digest Initialization

- The algorithm uses eight constants for message digest initialization. We call these constants A₀ to H₀ to match with the word naming used for the digest.
- Table 12.2 shows the value of these constants.

Table 12.2 *Values of constants in message digest initialization of SHA-512*

<i>Buffer</i>	<i>Value (in hexadecimal)</i>	<i>Buffer</i>	<i>Value (in hexadecimal)</i>
A ₀	6A09E667F3BCC908	E ₀	510E527FADE682D1
B ₀	BB67AE8584CAA73B	F ₀	9B05688C2B3E6C1F
C ₀	3C6EF372EF94F82B	G ₀	1F83D9ABFB41BD6B
D ₀	A54FE53A5F1D36F1	H ₀	5BE0CD19137E2179

- The values are calculated from the first eight prime numbers (2, 3, 5, 7, 11, 13, 17, and 19).
- Each value is the fraction part of the square root of the corresponding prime number after converting to binary and keeping only the first 64 bits.
- For example, the eighth prime is 19, with the square root $(19)^{1/2} = 4.35889894354$. Converting the number to binary with only 64 bits in the fraction part, we get

$(100.0101\ 1011\ 1110\ \dots\ 1001)_2 \rightarrow (4.5BE0CD19137E2179)_{16}$

CRYPTOGRAPHY

Hash Functions and Digital Signature- Unit 4:

Archana M

Department of Computer Applications

Message Digest (MD)

- Several hash algorithms were designed by Ron Rivest. These are referred to as MD2, MD4, and MD5, where MD stands for Message Digest.
- The last version, MD5, is a strengthened version of MD4 that divides the message into blocks of 512 bits and creates a 128-bit digest.
- It turned out that a message digest of size 128 bits is too small to resist collision attack.

MD5- Steps

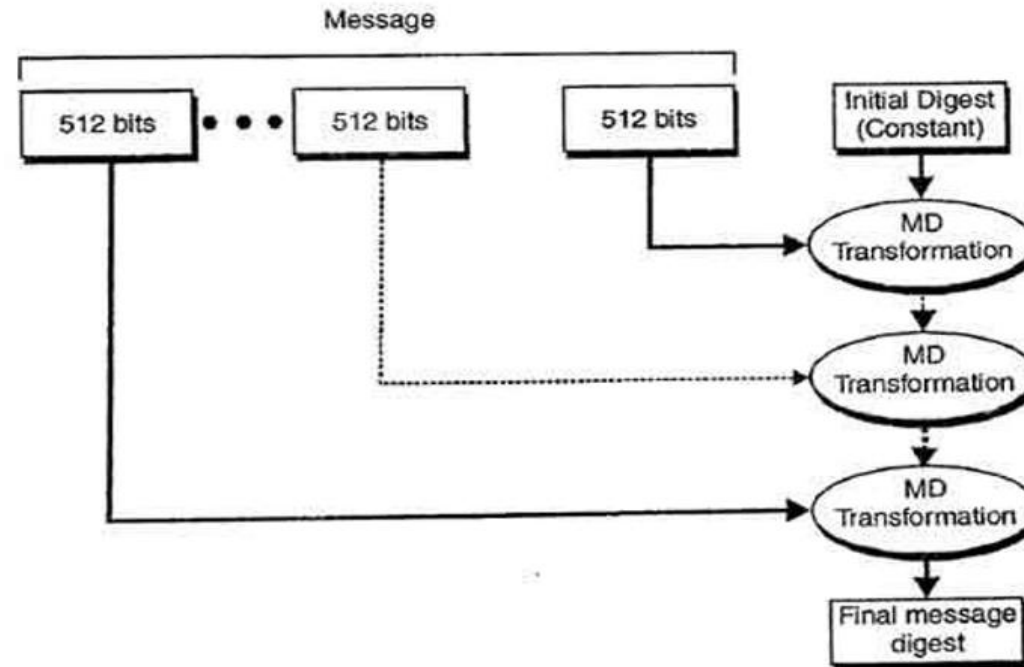
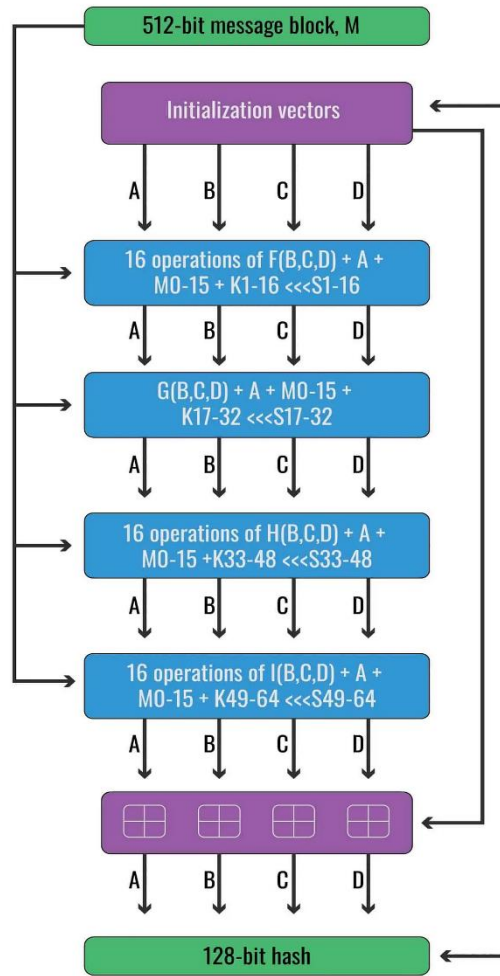
Input → any data (text, file, password)

Output → **128-bit hash** (usually shown as 32 hexadecimal characters)

Example:

Input: hello

MD5: 5d41402abc4b2a76b9719d911017c592



Generation of message digest using MD5.

Step 1: Padding :Message is padded to make its length a multiple of 512 bits

Step 2: Divide into Blocks :Split into 512-bit chunks

Step 3: Initialize Buffers :4 internal variables (A, B, C, D)

Step 4: Compression Function :Each block goes through 64 operations (rounds)

Step 5: Output :Final result → 128-bit hash

Key Properties

- Deterministic (same input → same output)
- Fast to compute
- Fixed output size (128 bits)
- Avalanche effect (small change → big difference)

MD5 is NOT Secure

- MD5 is now considered **broken** due to:
 - Collisions : Two different inputs can produce the **same hash**
 - Vulnerable to Attacks : Collision attacks , and Preimage weaknesses (partially)

Secure Hash Algorithm (SHA)



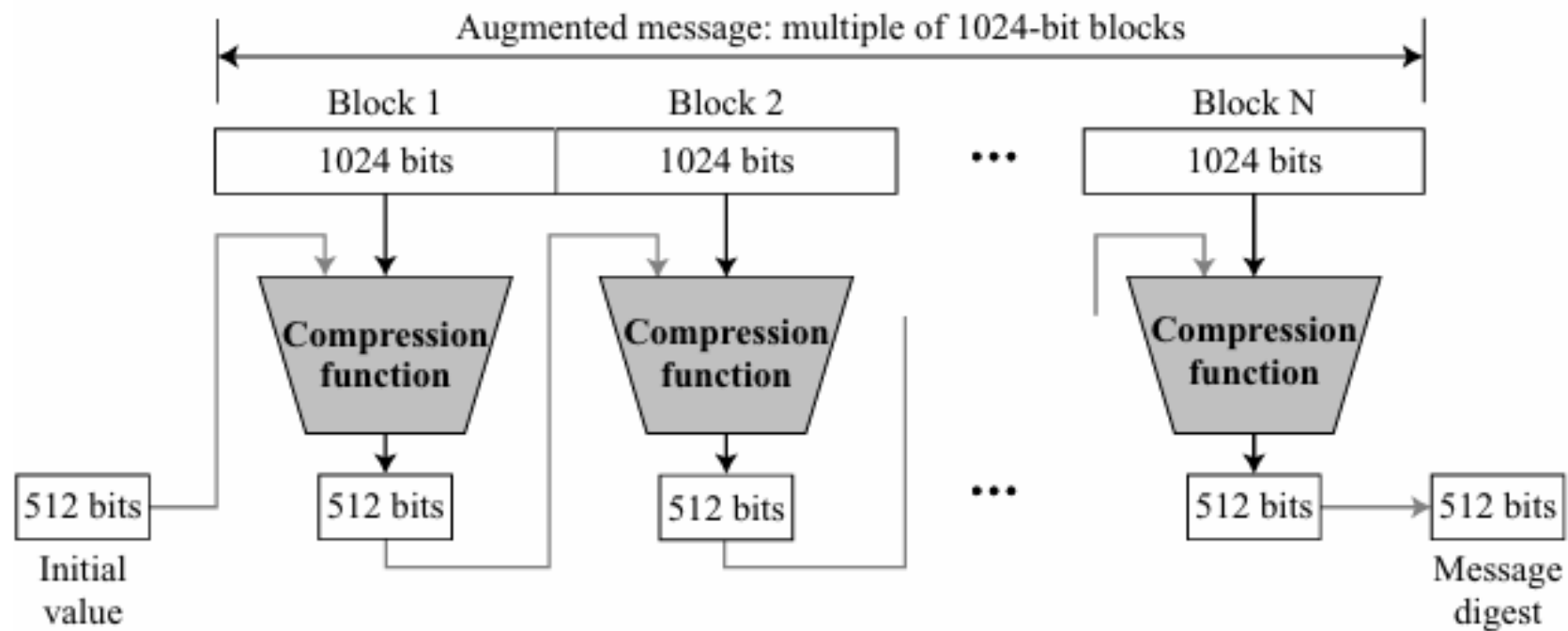
- The Secure Hash Algorithm (SHA) is a standard that was developed by the National Institute of Standards and Technology (NIST) and published as a Federal Information Processing standard (FIP 180).
- It is sometimes referred to as Secure Hash Standard (SHS).
- The standard is mostly based on MD5. The standard was revised in 1995 under FIP 180-1, which includes SHA-1
- It was revised later under FIP 180-2, which defines four new versions: SHA-224, SHA-256, SHA-384, and SHA-512

Table 12.1 *Characteristics of Secure Hash Algorithms (SHAs)*

<i>Characteristics</i>	<i>SHA-1</i>	<i>SHA-224</i>	<i>SHA-256</i>	<i>SHA-384</i>	<i>SHA-512</i>
Maximum Message size	$2^{64} - 1$	$2^{64} - 1$	$2^{64} - 1$	$2^{128} - 1$	$2^{128} - 1$
Block size	512	512	512	1024	1024
Message digest size	160	224	256	384	512
Number of rounds	80	64	64	80	80
Word size	32	32	32	64	64

-
- SHA-512 is the version of SHA with a 512-bit message digest
 - This version, like the others in the SHA family of algorithms, is based on the Merkle-Damgard scheme. It has a more complex structure than the others, and its message digest is the longest.
 - SHA-512 creates a digest of 512 bits from a multiple-block message. Each block is 1024 bits in length, as shown in Figure 12.6.

Figure 12.6 *Message digest creation SHA-512*



-
- The digest is initialized to a predetermined value of 512 bits.
 - The algorithm mixes this initial value with the first block of the message to create the first intermediate message digest of 512 bits.
 - This digest is then mixed with the second block to create the second intermediate digest.
 - Finally, the $(N - 1)$ th digest is mixed with the Nth block to create the Nth digest.
 - When the last block is processed, the resulting digest is the message digest for the entire message.

Message Preparation

- SHA-512 insists that the length of the original message be less than 2^{128} bits.
- This means that if the length of a message is equal to or greater than 2^{128} , it will not be processed by SHA-512.
- This is not usually a problem because 2^{128} bits is probably larger than the total storage capacity of any system

Example 12.1

This example shows that the message length limitation of SHA-512 is not a serious problem. Suppose we need to send a message that is 2^{128} bits in length. How long does it take for a communications network with a data rate of 2^{64} bits per second to send this message?

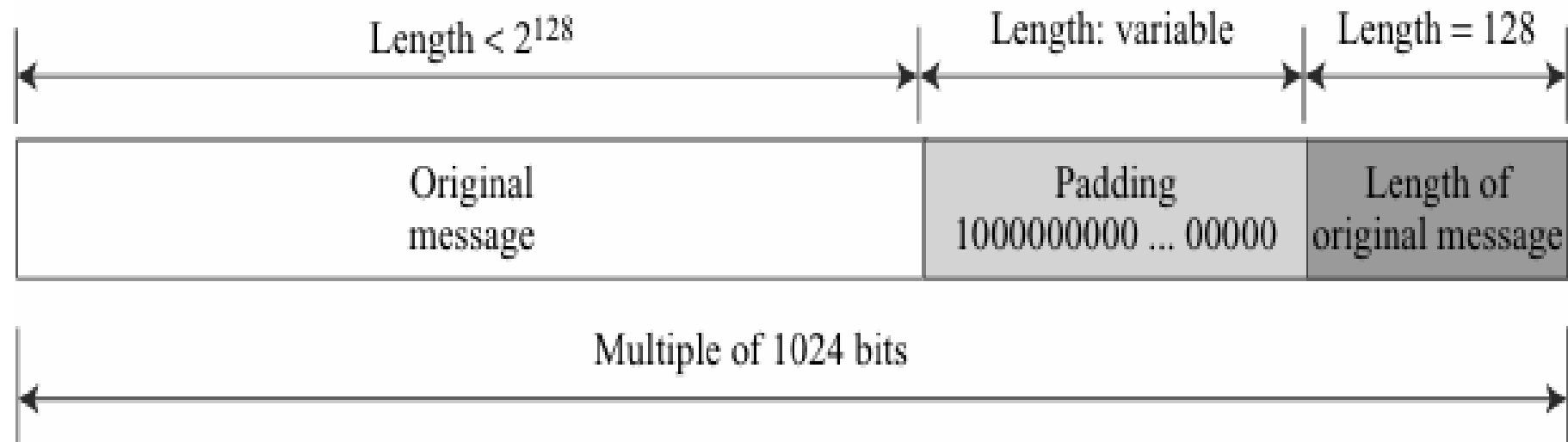
Solution

A communications network that can send 2^{64} bits per second is not yet available. Even if it were, it would take many years to send this message. This tells us that we do not need to worry about the SHA-512 message length restriction.

Length Field and Padding

- Before the message digest can be created, SHA-512 requires the addition of a 128-bit unsigned-integer length field to the message that defines the length of the message in bits.
- This is the length of the original message before padding.
- An unsigned integer field of 128 bits can define a number between 0 and $2^{128} - 1$.
- which is the maximum length of the message allowed in SHA-512. The length field defines the length of the original message before adding the length field or the padding

Figure 12.7 *Padding and length field in SHA-512*



Before the addition of the length field, we need to pad the original message to make the length a multiple of 1024. We reserve 128 bits for the length field, as shown in Figure 12.7. The length of the padding field can be calculated as follows. Let $|M|$ be the length of the original message and $|P|$ be the length of the padding field.

$$(|M| + |P| + 128) = 0 \bmod 1024 \quad \rightarrow \quad |P| = (-|M| - 128) \bmod 1024$$

The format of the padding is one 1 followed by the necessary number of 0s.

Example 12.3

What is the number of padding bits if the length of the original message is 2590 bits?

Solution

We can calculate the number of padding bits as follows:

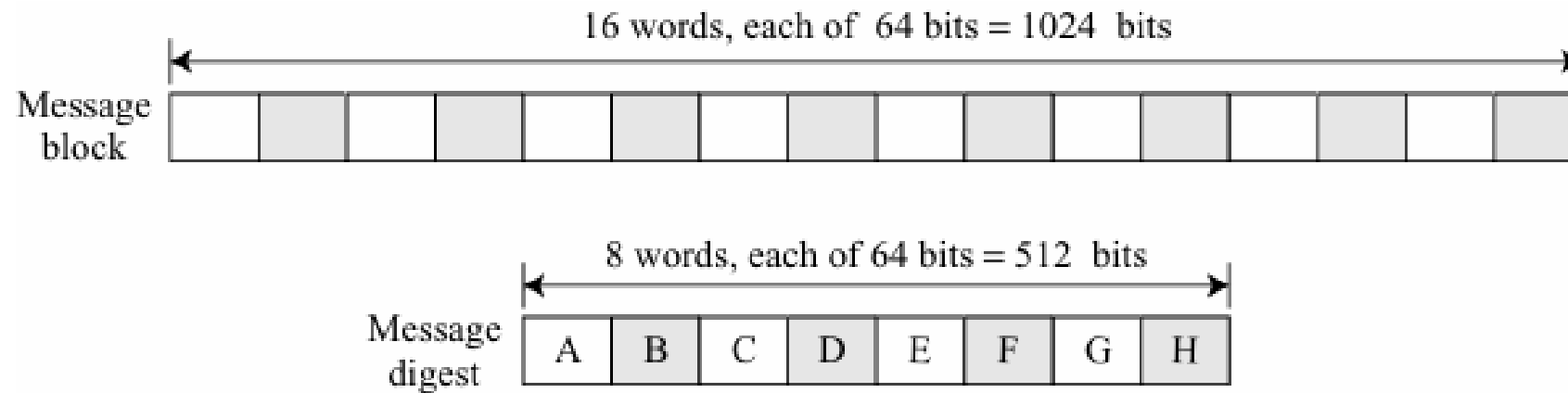
$$|P| = (-2590 - 128) \bmod 1024 = -2718 \bmod 1024 = 354$$

The padding consists of one 1 followed by 353 0's.

Words

- SHA-512 operates on words;
- it is word oriented. A word is defined as 64 bits.
- This means that, after the padding and the length field are added to the message, each block of the message consists of sixteen 64-bit words.
- The message digest is also made of 64-bit words, but the message digest is only eight words and the words are named A, B, C, D, E, F, G, and H, as shown in Figure

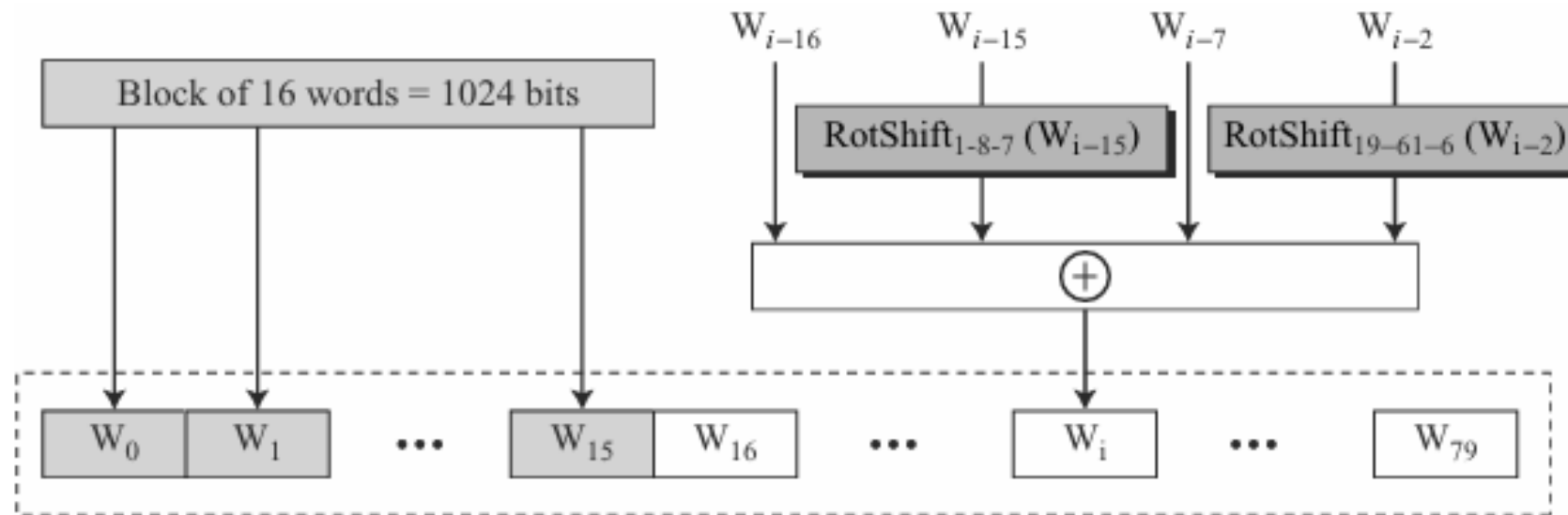
Figure 12.8 *A message block and the digest as words*



Word Expansion

- Before processing, each message block must be expanded. A block is made of 1024 bits, or sixteen 64-bit words.
- As we will see later, we need 80 words in the processing phase. So the 16-word block needs to be expanded to 80 words, from W .
- Figure 12.9 shows the word-expansion process.
- The 1024-bit block becomes the first 16 words;
- the rest of the words come from already-made words according to the operation shown in the figure.

Figure 12.9 *Word expansion in SHA-512*



$\text{RotShift}_{1-m-n}(x)$: $\text{RotR}_l(x) \oplus \text{RotR}_m(x) \oplus \text{ShL}_n(x)$

$\text{RotR}_i(x)$: Right-rotation of the argument x by i bits

$\text{ShL}_i(x)$: Shift-left of the argument x by i bits and padding the left by 0's.

Message Digest Initialization

- The algorithm uses eight constants for message digest initialization. We call these constants A_0 to H_0 to match with the word naming used for the digest.
- Table 12.2 shows the value of these constants.

Table 12.2 *Values of constants in message digest initialization of SHA-512*

<i>Buffer</i>	<i>Value (in hexadecimal)</i>	<i>Buffer</i>	<i>Value (in hexadecimal)</i>
A_0	6A09E667F3BCC908	E_0	510E527FADE682D1
B_0	BB67AE8584CAA73B	F_0	9B05688C2B3E6C1F
C_0	3C6EF372EF94F82B	G_0	1F83D9ABFB41BD6B
D_0	A54FE53A5F1D36F1	H_0	5BE0CD19137E2179

-
- The values are calculated from the first eight prime numbers (2, 3, 5, 7, 11, 13, 17, and 19).
 - Each value is the fraction part of the square root of the corresponding prime number after converting to binary and keeping only the first 64 bits.
 - For example, the eighth prime is 19, with the square root $(19)^{1/2} = 4.35889894354$. Converting the number to binary with only 64 bits in the fraction part, we get

$(100.0101\ 1011\ 1110\ \dots\ 1001)_2 \rightarrow (4.5BE0CD19137E2179)_{16}$

CRYPTOGRAPHY

Hash Functions and Digital Signature- Unit 4:

Archana M

Department of Computer Applications

Digital Signature



- We are all familiar with the concept of a signature. A person signs a document to show that it originated from her or was approved by her.
- The signature is proof to the recipient that the document comes from the correct entity.
- In other words, a signature on a document, when verified, is a sign of authentication the document is authentic.

Example: Consider a painting signed by an artist. The signature on the art, if authentic, means that the painting is probably authentic.

-
- When Alice sends a message to Bob, Bob needs to check the authenticity of the sender;
 - he needs to be sure that the message comes from Alice and not Eve.
 - Bob can ask Alice to sign the message electronically.
 - In other words, an electronic signature can prove the authenticity of Alice as the sender of the message.
 - We refer to this type of signature as a ***Digital Signature.***

COMPARISON- Differences between conventional signatures and digital signatures.



Inclusion

- A conventional signature is included in the document; it is part of the document.
- When we write a check, the signature is on the check; it is not a separate document.
- But when we sign a document digitally, we send the signature as a separate document.
- The sender sends two documents: the message and the signature.
- The recipient receives both documents and verifies that the signature belongs to the supposed sender.
- If this is proven, the message is kept; otherwise, it is rejected.

Verification Method

- The second difference between the two types of signatures is the method of verifying the signature.
- For a conventional signature, when the recipient receives a document, she compares the signature on the document with the signature on file.
- If they are the same, the document is authentic.
- The recipient needs to have a copy of this signature on file for comparison.
- For a digital signature, the recipient receives the message and the signature.
- A copy of the signature is not stored anywhere.
- The recipient needs to apply a verification technique to the combination of the message and the signature to verify the authenticity.

Relationship

- For a conventional signature, there is normally a one-to-many relationship between a signature and documents.
- A person uses the same signature to sign many documents.
- For a digital signature, there is a one-to-one relationship between a signature and a message. Each message has its own signature.
- The signature of one message cannot be used in another message.
- If Bob receives two messages, one after another, from Alice, he cannot use the signature of the first message to verify the second.
- Each message needs a new signature

Duplicity

- Another difference between the two types of signatures is a quality called duplicity.
- In conventional signature, a copy of the signed document can be distinguished from the original one on file.
- In digital signature, there is no such distinction unless there is a factor of time (such as a timestamp) on the document.
- For example, suppose Alice sends a document instructing Bob to pay Eve.
- If Eve intercepts the document and the signature, she can replay it later to get money again from Bob.

Aspect	Conventional Signature	Digital Signature
Definition	A handwritten mark used to verify identity on physical documents	A cryptographic technique used to verify authenticity and integrity of digital data
Form	Physical (ink on paper)	Electronic (generated using algorithms)
Verification	Verified visually by humans	Verified using mathematical algorithms and public key infrastructure (PKI)
Security Level	Low (can be forged or copied)	High (uses encryption, very difficult to forge)
Authentication	Confirms identity based on appearance	Confirms identity using private/public key pair
Integrity of Data	Does not ensure document content hasn't changed	Ensures data integrity (any change invalidates signature)

Aspect	Conventional Signature	Digital Signature
Non-repudiation	Weak (person can deny signing)	Strong (signer cannot deny due to cryptographic proof)
Dependency	Depends on human judgment	Depends on cryptographic algorithms and software
Speed	Slow (manual process)	Fast (instant verification)
Usage Area	Paper-based documents (contracts, checks)	Digital documents (emails, e-transactions, software distribution)
Storage	Requires physical storage	Stored electronically

CRYPTOGRAPHY

Hash Functions and Digital Signature- Unit 4:

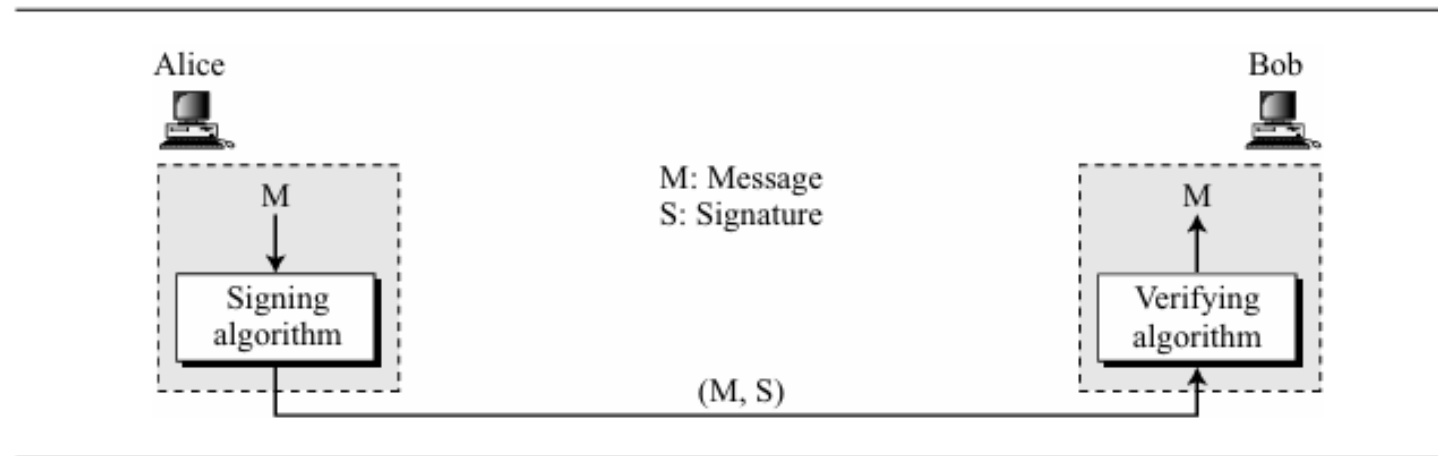
Archana M

Department of Computer Applications

Digital signature process

- The sender uses a signing algorithm to sign the message.
- The message and the signature are sent to the receiver.
- The receiver receives the message and the signature and applies the verifying algorithm to the combination.
- If the result is true, the message is accepted; otherwise, it is rejected.

Figure 13.1 *Digital signature process*

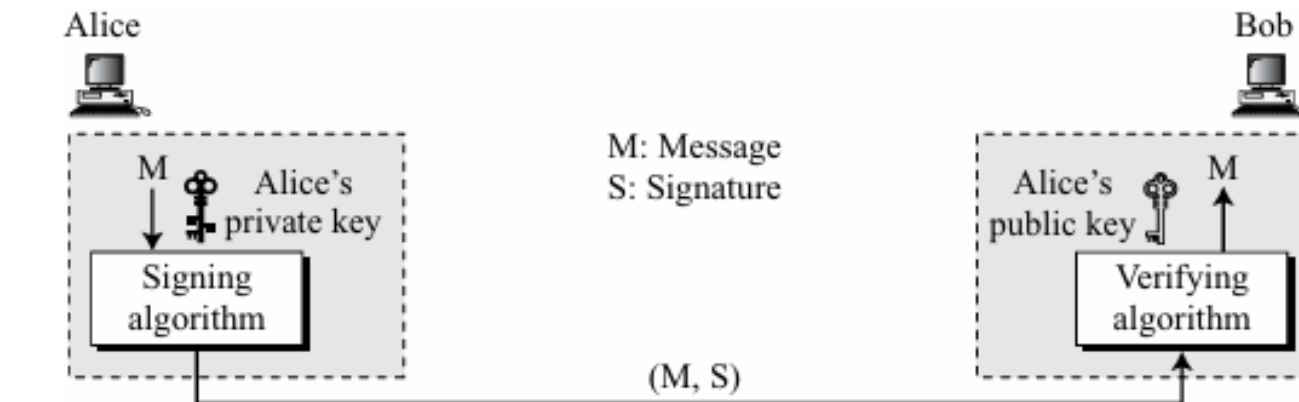


Need for Keys

- A conventional signature is like a private “key” belonging to the signer of the document.
- The signer uses it to sign documents; no one else has this signature.
- The copy of the signature is on file like a public key;
- Anyone can use it to verify a document, to compare it to the original signature.
- In a digital signature, the signer uses her private key, applied to a signing algorithm, to sign the document.
- The verifier, on the other hand, uses the public key of the signer, applied to the verifying algorithm, to verify the document.

- Note that when a document is signed, anyone, including Bob, can verify it because everyone has access to Alice's public key.
- Alice must not use her public key to sign the document because then anyone could forge her signature

Figure 13.2 *Adding key to the digital signature process*



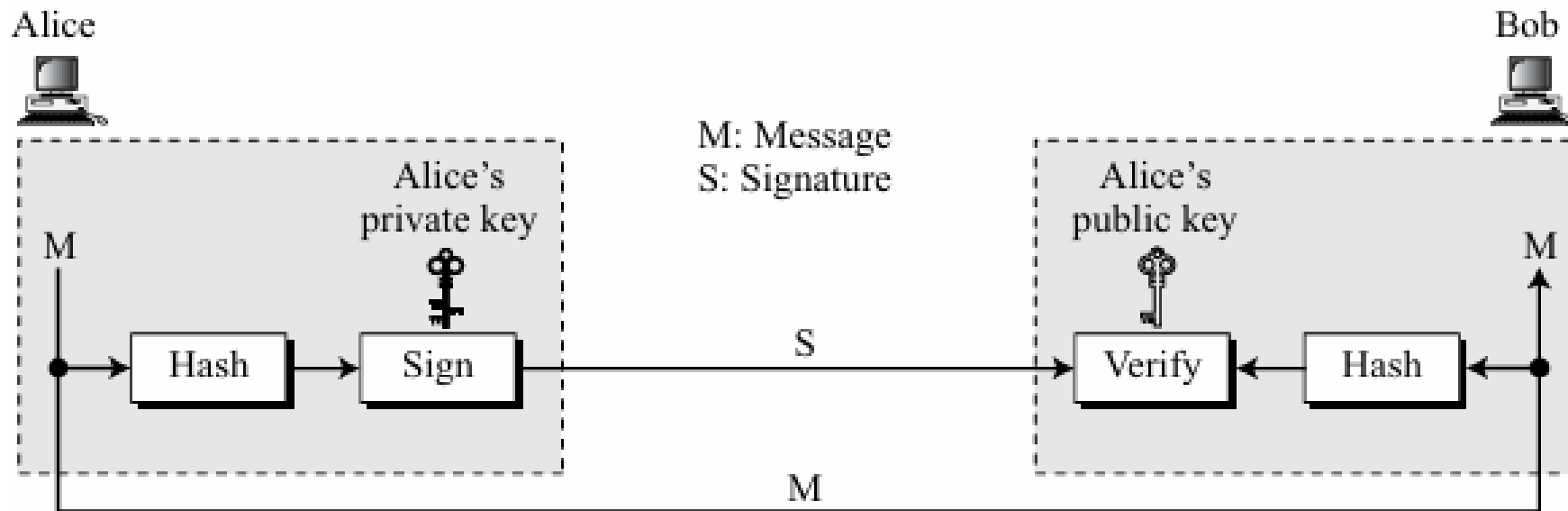
A digital signature needs a public-key system.

The signer signs with her private key; the verifier verifies with the signer's public key.

Signing the Digest

- In a digital signature system, the messages are normally long, but we have to use asymmetric-key schemes.
- The solution is to sign a digest of the message, which is much shorter than the message.
- a carefully selected message digest has a one-to-one relationship with the message.
- The sender can sign the message digest and the receiver can verify the message digest.
- The effect is the same. Figure 13.3 shows signing a digest in a digital signature system.

Figure 13.3 *Signing the digest*



-
- A digest is made out of the message at Alice's site.
 - The digest then goes through the signing process using Alice's private key.
 - Alice then sends the message and the signature to Bob.
 - At Bob's site, using the same public hash function, a digest is first created out of the received message.
 - Calculations are done on the signature and the digest.
 - The verifying process also applies criteria on the result of the calculation to determine the authenticity of the signature.
 - If authentic, the message is accepted; otherwise, it is rejected.

CRYPTOGRAPHY

Hash Functions and Digital Signature- Unit 4:

Archana M

Department of Computer Applications

DIGITAL SIGNATURE SERVICES

- **Digital Signature Services (DSS)** are systems or platforms that allow users to **create, manage, and verify digital signatures** on electronic documents securely using cryptography.
- A Digital Signature Service acts like a **trusted digital notary**:
 - Confirms *who* signed the document
 - Ensures the document was *not altered*
 - Provides *legal proof* of signing

Functions of Digital Signature Services

- **Signature Creation**

- Uses a private key to sign documents digitally

- **Signature Verification**

- Uses a public key to verify authenticity

- **Certificate Management**

- Issues and manages digital certificates via a Certificate Authority

- **Timestamping**

- Adds time proof to prevent disputes

- **Encryption Support**

- Secures the data during transmission

SERVICES- Message Authentication

- A secure digital signature scheme, like a secure conventional signature (one that cannot be easily copied) can provide message authentication (also referred to as data-origin authentication).
- Bob can verify that the message is sent by Alice because Alice's public key is used in verification.
- Alice's public key cannot verify the signature signed by Eve's private key.

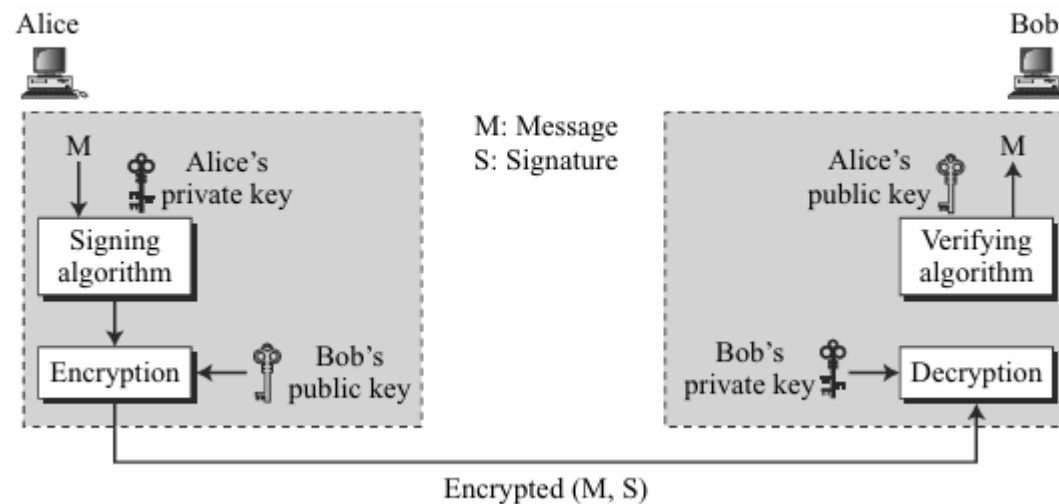
SERVICES- Message INTEGRITY

- The integrity of the message is preserved even if we sign the whole message because we cannot get the same signature if the message is changed.
- The digital signature schemes today use a hash function in the signing and verifying algorithms that preserve the integrity of the message

SERVICES- Confidentiality

- A digital signature does not provide confidential communication.
- If confidentiality is required, the message and the signature must be encrypted using either a secret-key or public-key cryptosystem.
- Figure 13.5 shows how this extra level can be added to a simple digital signature scheme.

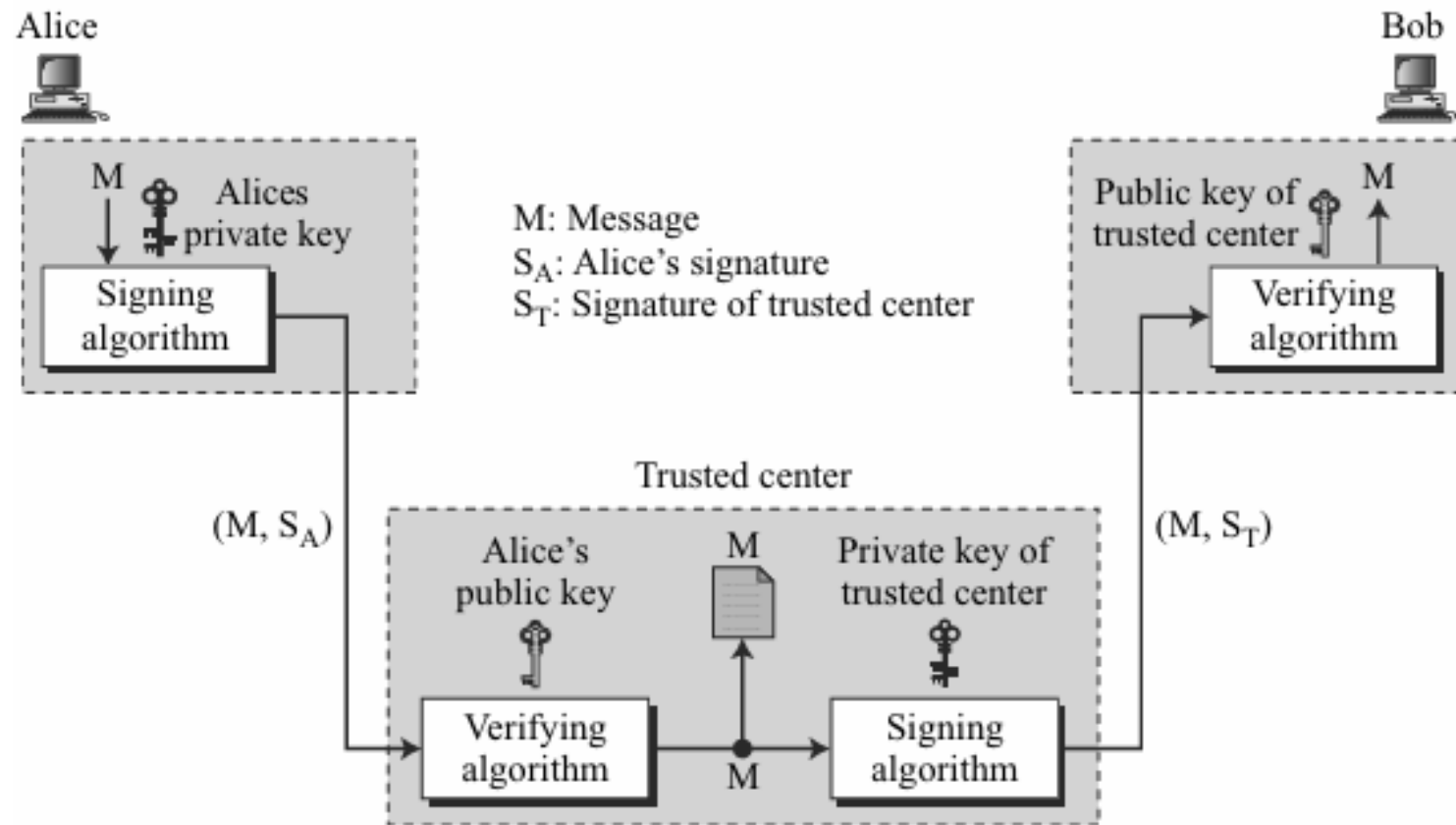
Figure 13.5 *Adding confidentiality to a digital signature scheme*



SERVICES - Nonrepudiation

- If Alice signs a message and then denies it, can Bob later prove that Alice actually signed it?
- One solution is a trusted third party. People can create an established trusted party among themselves
- Alice creates a signature from her message (S_A) and sends the message, her identity, Bob's identity, and the signature to the center.
- The center, after checking that Alice's public key is valid, verifies through Alice's public key that the message came from Alice.
- The center then saves a copy of the message with the sender identity, recipient identity, and a timestamp in its archive.
- The center uses its private key to create another signature (S_T) from the message.
- The center then sends the message, the new signature, Alice's identity, and Bob's identity to Bob. Bob verifies the message using the public key of the trusted center

Figure 13.4 *Using a trusted center for nonrepudiation*



CRYPTOGRAPHY

Hash Functions and Digital Signature- Unit 4:

Archana M

Department of Computer Applications

ATTACKS ON DIGITAL SIGNATURE

Attack Types - three kinds of attacks on digital signatures:

- key-only,
- known-message,
- and chosen-message.

-
- Key-Only Attack
 - In the key-only attack, Eve has access only to the public information released by Alice.
 - To forge a message, Eve needs to create Alice's signature to convince Bob that the message is coming from Alice.
 - This is the same as the ciphertext-only attack we discussed for encipherment.

-
- Known-Message Attack
 - In the known-message attack, Eve has access to one or more message-signature pairs.
 - In other words, she has access to some documents previously signed by Alice.
 - Eve tries to create another message and forge Alice's signature on it.
 - This is similar to the known-plaintext attack we discussed for encipherment.

-
- Chosen-Message Attack
 - In the chosen-message attack, Eve somehow makes Alice sign one or more messages for her.
 - Eve now has a chosen-message/signature pair.
 - Eve later creates another message, with the content she wants, and forges Alice's signature on it.
 - This is similar to the chosen-plaintext attack we discussed for encipherment.

Forgery Types

- If the attack is successful, the result is a forgery. We can have two types of forgery: existential and selective.
- Existential Forgery :In an existential forgery, Eve may be able to create a valid message-signature pair, but not one that she can really use.
- In other words, a document has been forged, but the content is randomly calculated.
- This type of forgery is probable, but fortunately Eve cannot benefit from it very much.
- Her message could be syntactically or semantically unintelligible

-
- Selective Forgery:
 - In selective forgery, Eve may be able to forge Alice's signature on a message with the content selectively chosen by Eve.
 - Although this is beneficial to Eve, and may be very detrimental to Alice, the probability of such forgery is low, but not negligible.

CRYPTOGRAPHY

Hash Functions and Digital Signature- Unit 4:

Archana M

Department of Computer Applications

DIGITAL SIGNATURE SCHEMES

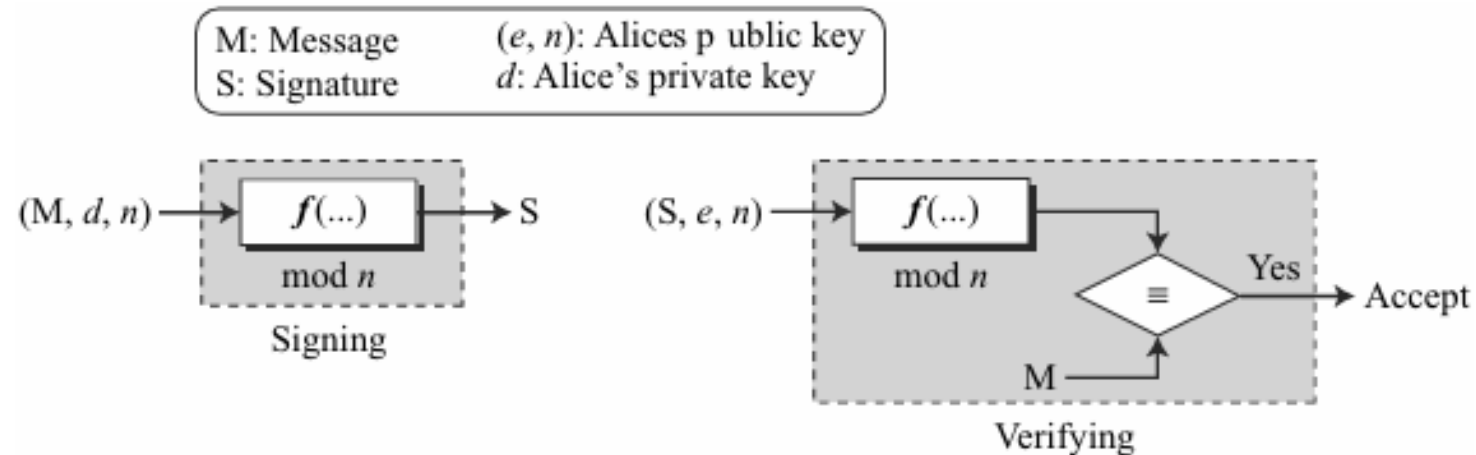
RSA Digital Signature Scheme

We have discussed how to use RSA cryptosystem to provide privacy.

- The RSA idea can also be used for signing and verifying a message.
- In this case, it is called the RSA digital signature scheme.
- The digital signature scheme changes the roles of the private and public keys.
- First, the private and public keys of the sender, not the receiver, are used.
- Second, the sender uses her own private key to sign the document; the receiver uses the sender's public key to verify it

- Figure 13.6 gives the general idea behind the RSA digital signature scheme

Figure 13.6 *General idea behind the RSA digital signature scheme*



The signing and verifying sites use the same function, but with different parameters. The verifier compares the message and the output of the function for congruence. If the result is true, the message is accepted.

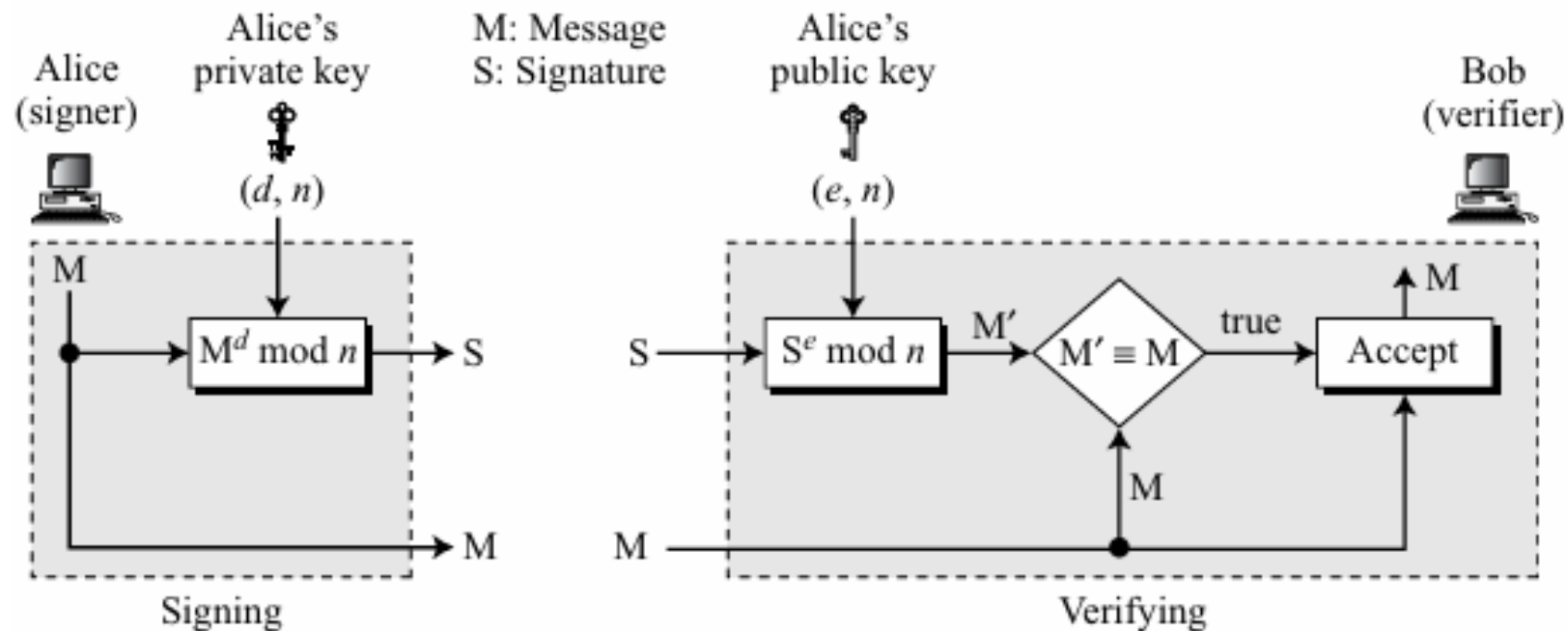
-
- Key Generation
 - Key generation in the RSA digital signature scheme is exactly the same as key generation in the RSA cryptosystem
 - Alice chooses two primes p and q and calculates $n = p \times q$. Alice calculates $\phi(n) = (p - 1)(q - 1)$.
 - She then chooses e , the public exponent, and calculates d , the private exponent such that $e \times d = 1 \bmod \phi(n)$.
 - Alice keeps d ; she publicly announces n and e

In the RSA digital signature scheme, d is private; e and n are public.

Signing and Verifying

Figure 13.7 shows the RSA digital signature scheme.

Figure 13.7 *RSA digital signature scheme*



Signing Alice creates a signature out of the message using her private exponent, $S = M^d \bmod n$ and sends the message and the signature to Bob.

Verifying Bob receives M and S . Bob applies Alice's public exponent to the signature to create a copy of the message $M' = S^e \bmod n$. Bob compares the value of M' with the value of M . If the two values are congruent, Bob accepts the message. To prove this, we start with the verification criteria:

$$M' \equiv M \pmod{n} \quad \rightarrow \quad S^e \equiv M \pmod{n} \quad \rightarrow \quad M^{d \times e} \equiv M \pmod{n}$$

Example

Example 13.1

For the security of the signature, the value of p and q must be very large. As a trivial example, suppose that Alice chooses $p = 823$ and $q = 953$, and calculates $n = 784319$. The value of $\phi(n)$ is 782544. Now she chooses $e = 313$ and calculates $d = 160009$. At this point key generation is complete. Now imagine that Alice wants to send a message with the value of $M = 19070$ to Bob. She uses her private exponent, 160009, to sign the message:

$$M: 19070 \rightarrow S = (19070^{160009}) \bmod 784319 = 210625 \bmod 784319$$

Alice sends the message and the signature to Bob. Bob receives the message and the signature. He calculates

$$M' = 210625^{313} \bmod 784319 = 19070 \bmod 784319 \quad \rightarrow \quad M \equiv M' \bmod n$$

Bob accepts the message because he has verified Alice's signature.

CRYPTOGRAPHY

Hash Functions and Digital Signature- Unit 4:

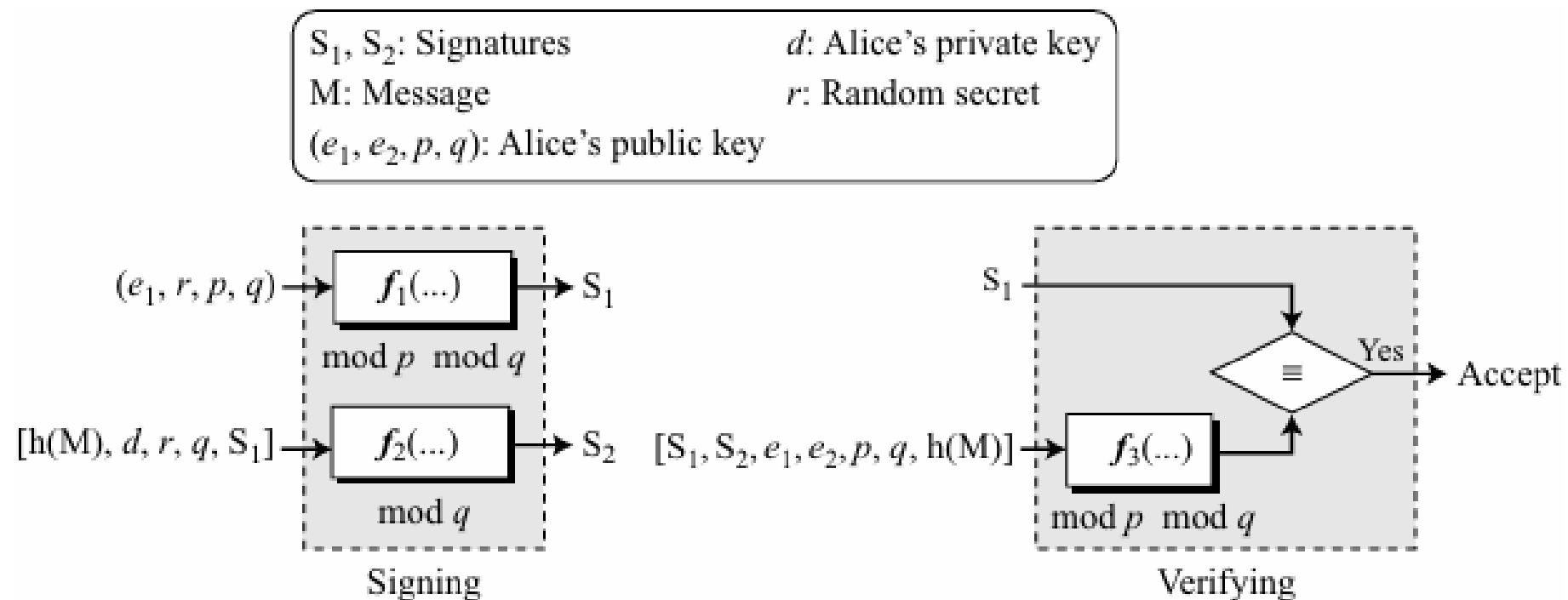
Archana M

Department of Computer Applications

Digital Signature Standard (DSS)

- The Digital Signature Standard (DSS) was adopted by the National Institute of Standards and Technology (NIST) in 1994.
- NIST published DSS as FIPS 186. DSS uses a digital signature algorithm (DSA) based on the ElGamal scheme with some ideas from the Schnorr scheme.
- DSS has been criticized from the time it was published. The main complaint regards the secrecy of DSS design.
- The second complaint regards the size of the prime, 512 bits. Later NIST made the size variable to respond to this complaint.

Figure 13.13 *General idea behind DSS scheme*



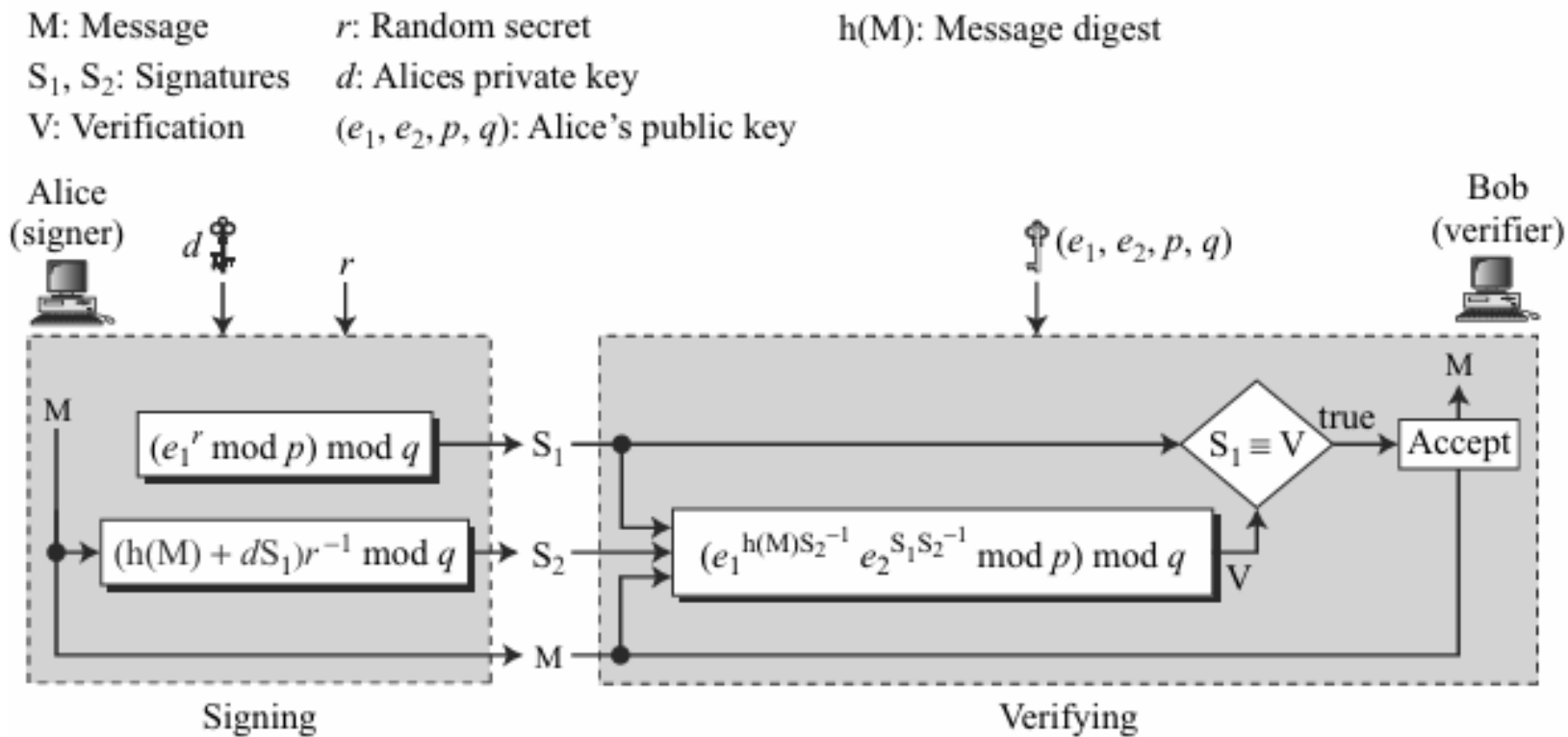
-
- In the signing process, two functions create two signatures;
 - in the verifying process, the output of one function is compared to the first signature for verification.
 - This is similar to Schnorr, but the inputs are different. Another difference is that this scheme uses the message digest (not the message) as part of inputs to functions 1 and 3.
 - The interesting point is that the scheme uses two public moduli: p and q .
 - Functions 1 and 3 use both p and q ; function 2 uses only q .

- Key Generation :Before signing a message to any entity, Alice needs to generate keys and announce the public ones to the public.

1. Alice chooses a prime p , between 512 and 1024 bits in length. The number of bits in p must be a multiple of 64.
2. Alice chooses a 160-bit prime q in such a way that q divides $(p - 1)$.
3. Alice uses two multiplication groups $\langle \mathbf{Z}_p^*, \times \rangle$ and $\langle \mathbf{Z}_q^*, \times \rangle$; the second is a subgroup of the first.
4. Alice creates e_1 to be the q th root of 1 modulo p ($e_1^p = 1 \bmod p$). To do so, Alice chooses a primitive element in $\mathbf{Z}_p$, e_0 , and calculates $e_1 = e_0^{(p-1)/q} \bmod p$.
5. Alice chooses d as the private key and calculates $e_2 = e_1^d \bmod p$.
6. Alice's public key is (e_1, e_2, p, q) ; her private key is (d) .

Verifying and Signing

Figure 13.14 *DSS scheme*



Signing :The following shows the steps to sign the message

1. Alice chooses a random number r ($1 \leq r \leq q$). Note that although public and private keys can be chosen once and used to sign many messages, Alice needs to select a new r each time she needs to sign a new message.
2. Alice calculates the first signature $S_1 = (e_1^r \bmod p) \bmod q$. Note that the value of the first signature does not depend on M , the message.
3. Alice creates a digest of message $h(M)$.
4. Alice calculates the second signature $S_2 = (h(M) + d S_1)r^{-1} \bmod q$. Note that the calculation of S_2 is done in modulo q arithmetic.
5. Alice sends M , S_1 , and S_2 to Bob.

Verifying: Following are the steps used to verify the message when M , S_1 , and S_2 are received

1. Bob checks to see if $0 < S_1 < q$.
2. Bob checks to see if $0 < S_2 < q$.
3. Bob calculates a digest of M using the same hash algorithm used by Alice.
4. Bob calculates $V = [(e_1^{h(M)S_2^{-1}} e_2^{S_1S_2^{-1}}) \bmod p] \bmod q$.
5. If S_1 is congruent to V , the message is accepted; otherwise, it is rejected.

Example 13.5

Alice chooses $q = 101$ and $p = 8081$. Alice selects $e_0 = 3$ and calculates $e_1 = e_0^{(p-1)/q} \bmod p = 6968$. Alice chooses $d = 61$ as the private key and calculates $e_2 = e_1^d \bmod p = 2038$. Now Alice can send a message to Bob. Assume that $h(M) = 5000$ and Alice chooses $r = 61$:

$$h(M) = 5000 \quad r = 61$$

$$S_1 = (e_1^r \bmod p) \bmod q = 54$$

$$S_2 = ((h(M) + d S_1) r^{-1}) \bmod q = 40$$

Alice sends M , S_1 , and S_2 to Bob. Bob uses the public keys to calculate V .

$$S_2^{-1} = 48 \bmod 101$$

$$V = [(6968^{5000 \times 48} \times 2038^{54 \times 48}) \bmod 8081] \bmod 101 = 54$$

Because S_1 and V are congruent, Bob accepts the message.



THANK YOU

Archana M

Department of Computer Applications

archanam@pes.edu